

Juan Pablo Cuervo Contreras

Carné: 1000100656

David Felipe Ortiz Salcedo

Carné: 1000100448

ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO ORIENTADO POR OBJETOS

Persistencia 2026-1 — Laboratorio 6/6 [:)]

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Extender el código de un proyecto considerando nuevos requisitos funcionales.
2. Diseñar y construir los métodos básicos de manejo de archivos: guardar como, abrir, exportar como e importar.
3. Controlar las excepciones generadas al trabajar con archivos.
4. Experimentar las prácticas

Coding Only one pair [integrates code at a time](#).

Coding. Use [collective ownership](#).

ENTREGA

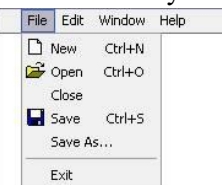
- Incluyan en un archivo **.zip** los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (Casallas-Duarte) Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO

Reto: Extender el proyecto [forest](#) adicionando el menú Abrir en el menú barra con las opciones básicas de entrada-salida (Abrir - Guardar como e Importar - Exportar como) y las opciones estándar Nuevo y Salir.

El menú Archivo reúne las funciones básicas de gestión de documentos organizadas de forma consistente y predecible.

Las guías de *Apple* y las directrices de *Microsoft* para el diseño de interfaz establecen que esta estructura y semántica deben mantenerse estables para garantizar la usabilidad, constituyendo así un estándar de facto en las interfaces gráficas modernas.



PARTE I. Creando la maqueta [En **lab06.doc forest.astah *.java**]

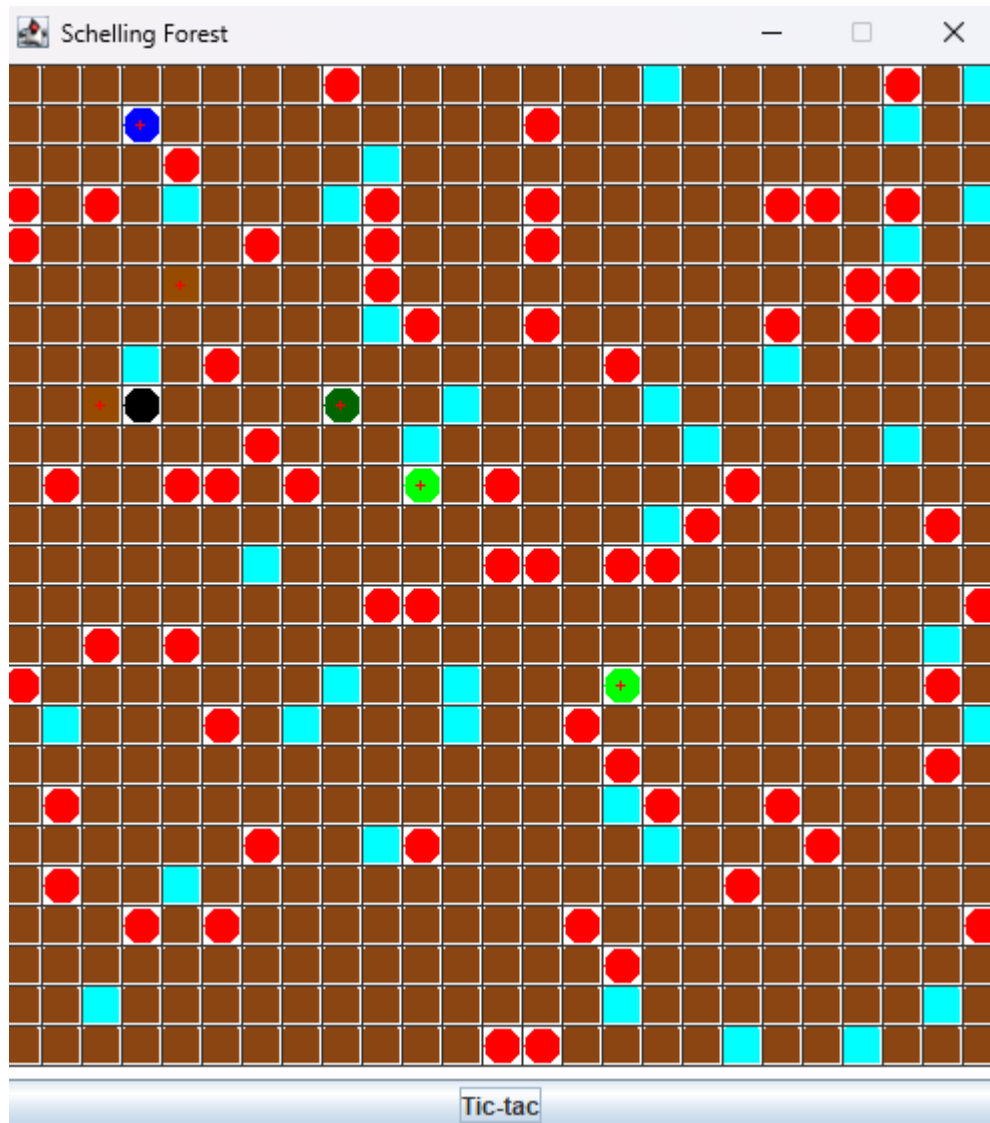
(NO OLVIDEN BDD – MDD)

En este punto vamos a construir la maqueta correspondiente a esta extensión siguiendo el patrón MVC.

A [forest](#) inicial

1. En su directorio descarguen el laboratorio 03 realizado por ustedes y seleccionen el ambiente para trabajar: [BlueJ](#) o [Eclipse](#). ¿Cuál ambiente seleccionaron? ¿Por qué?
R/: Elegimos BlueJ porque nos parece un ambiente más fácil de usar y entender a diferencia de Eclipse donde este editor exige mucho más durante el desarrollo de un código a pesar de que se enfoca en un ambiente más profesional.
2. Ejecuten el programa y capturen la pantalla.

R/:



B Maqueta: MVC

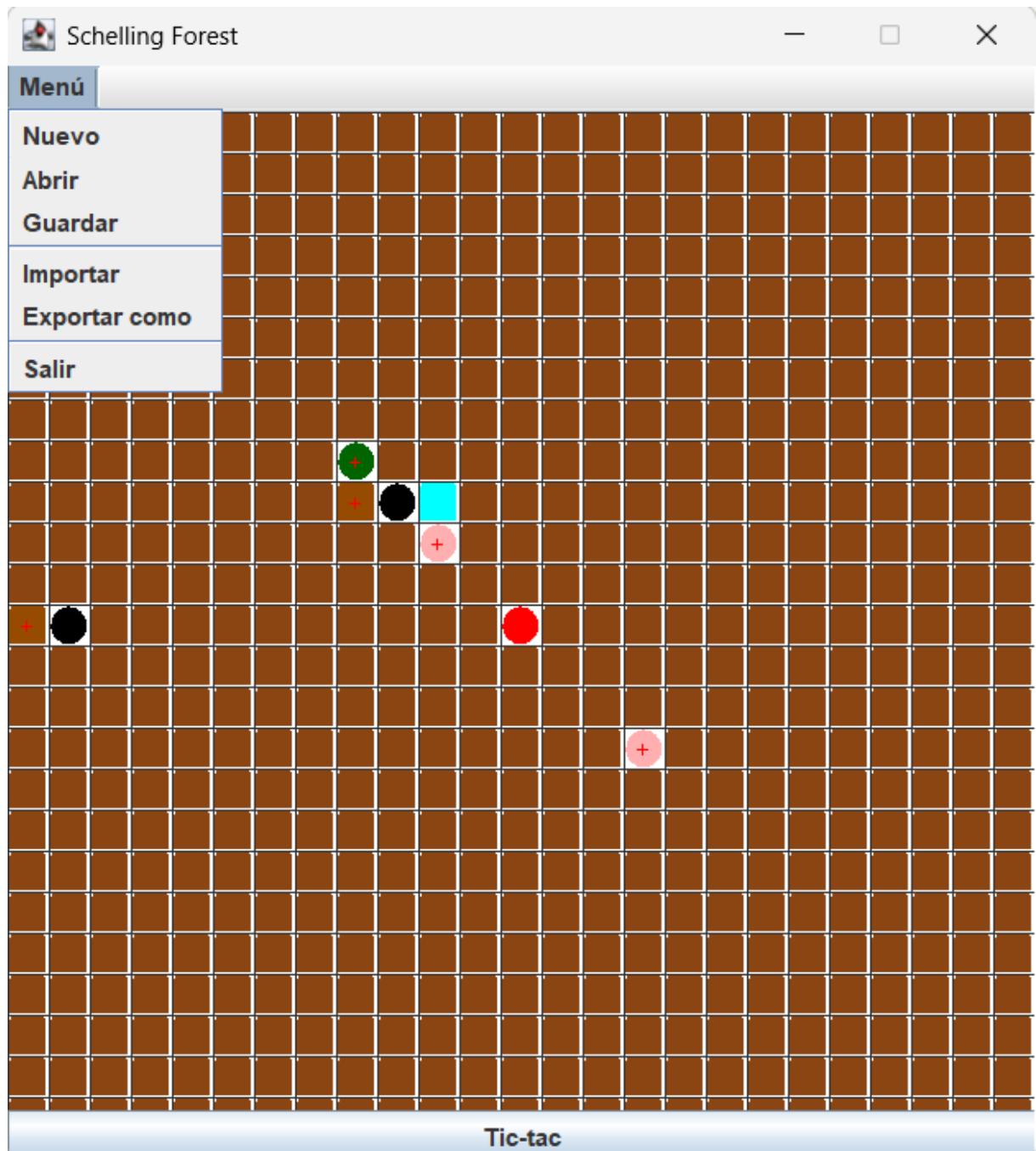
1. **MODELO:** Preparen en la clase fachada del dominio los métodos correspondientes a las cuatro opciones básicas de entrada-salida ([open](#), [saveAs](#), [import](#) y [exportAs](#)). Estos métodos deben tener un parámetro [File](#) y simplemente deben propagar una [Forest Exception](#) con el mensaje de “Opción nombreOpción en construcción. Archivo nombreArchivo”.

Para la opción Nuevo se usará el constructor de la clase [Forest](#) y la opción Salir no requiere del modelo.

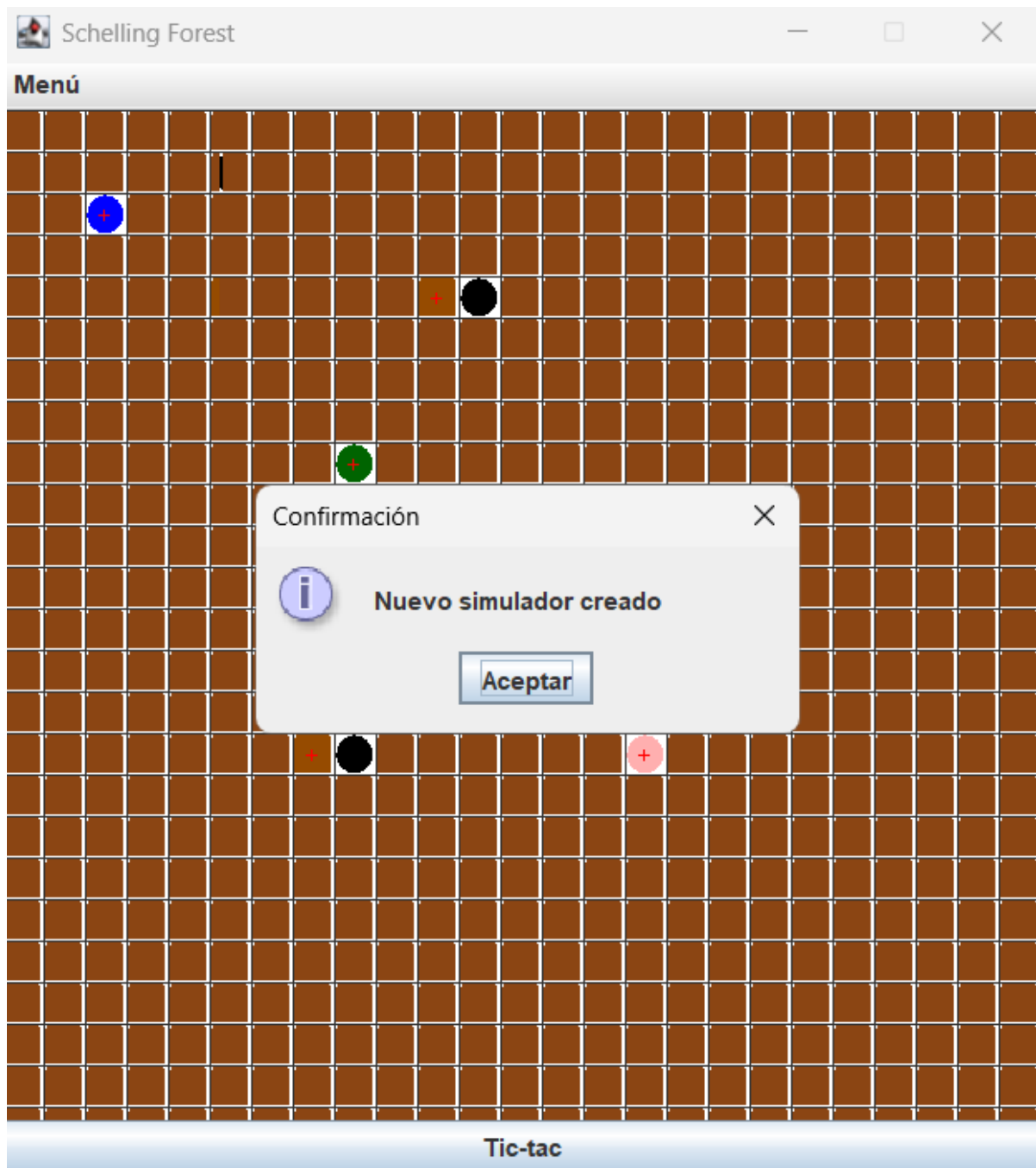
2. **VISTA:** Construyan el menú Archivo que ofrezca, además de las opciones básicas de entrada-salida, las opciones estándar de nuevo y salir (Nuevo, Abrir, Salvar como, Importar, Exportar como y Salir). No olviden incluir los separadores.

Para esto creen el método [prepareElementsMenu](#). Capturen la pantalla del menú.

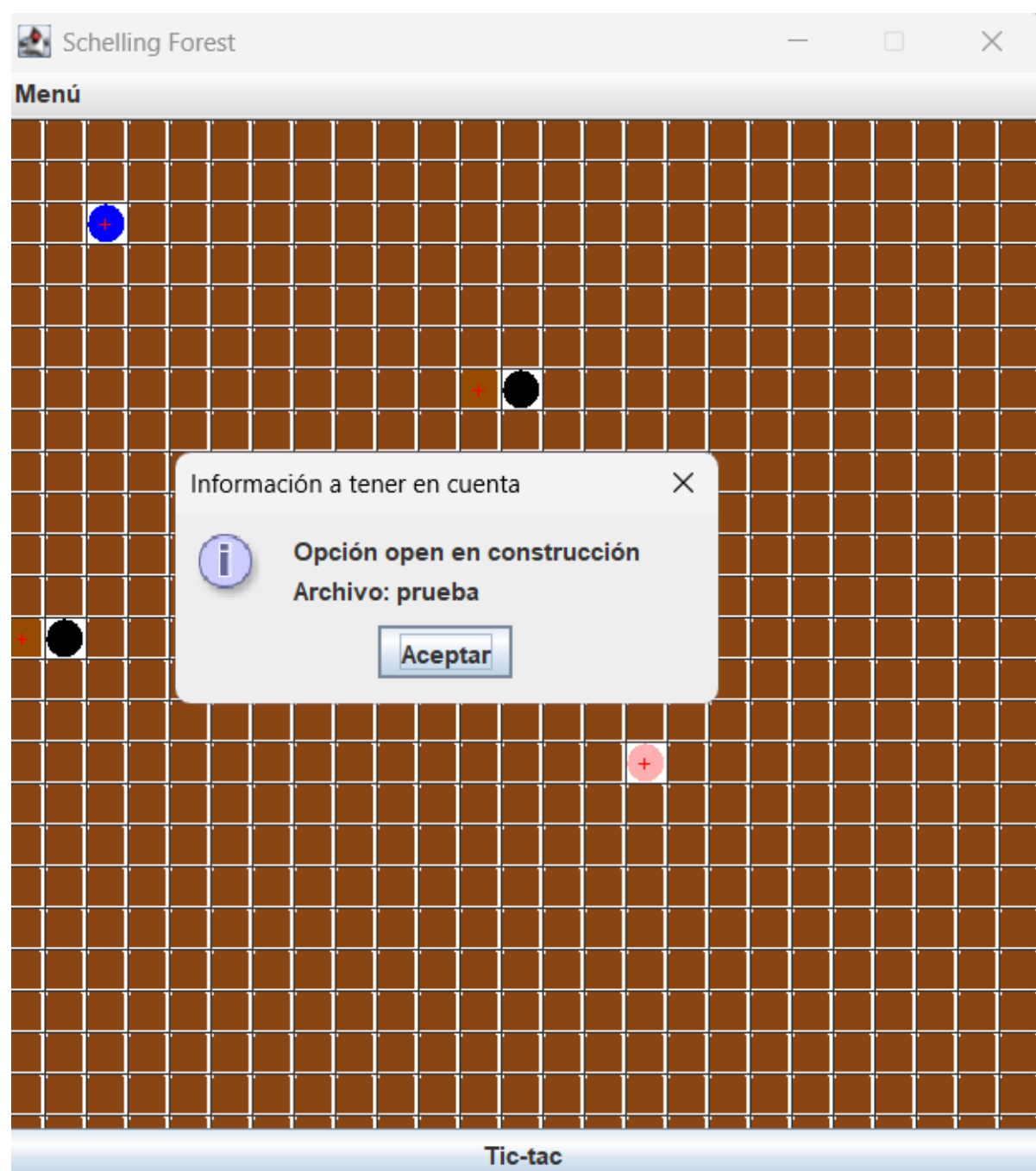
R/:

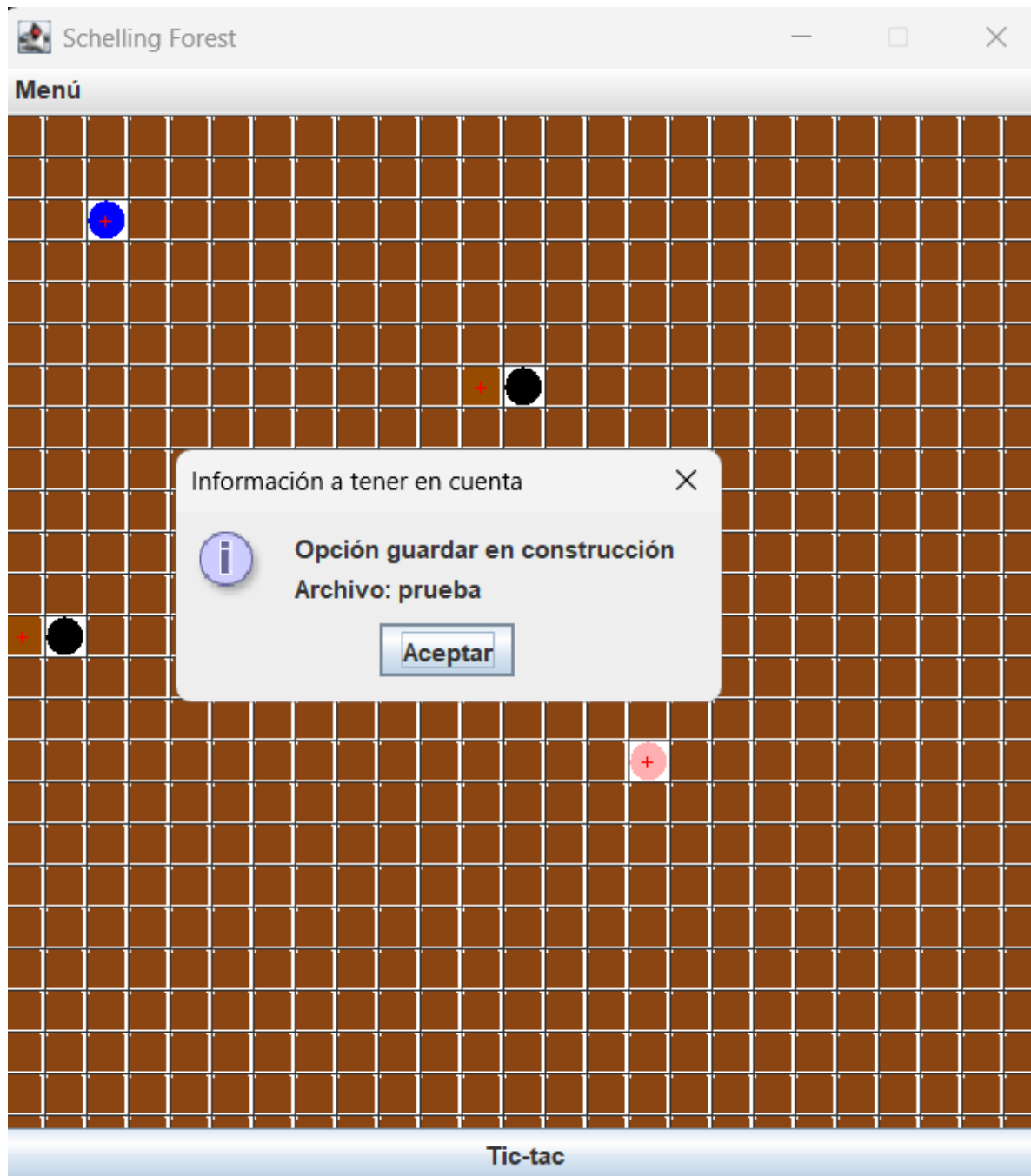


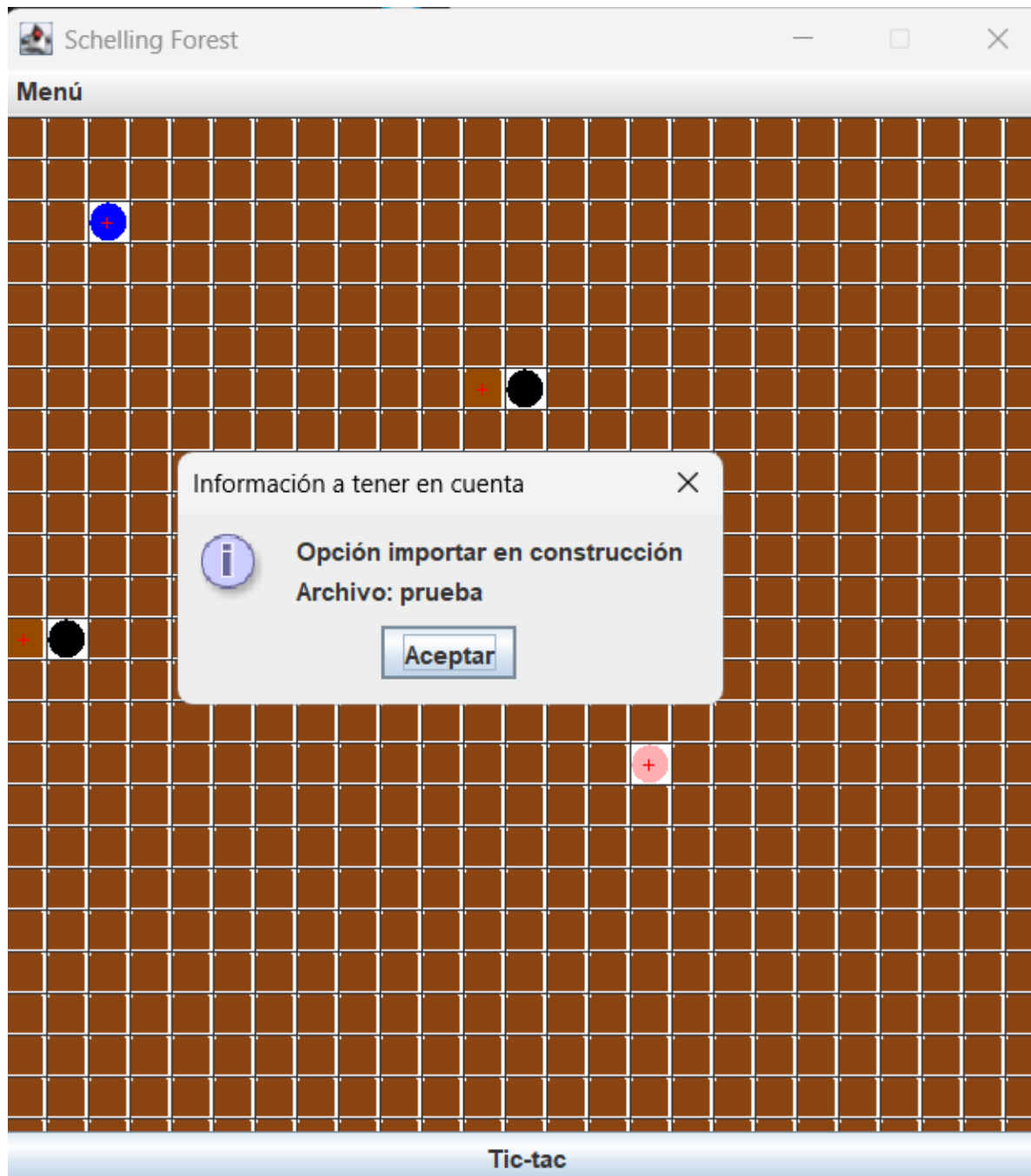
3. **CONTROLADOR:** Construyan los oyentes correspondientes a las seis opciones. Para esto creen el método `prepareActionsMenu` y los métodos base del controlador (`optionOpen`, `optionSaveAs`, `optionImport`, `optionExportAs`, `optionNew`, `optionExit`).
 - a) Salir: El método `optionExit` que hace que se termine la aplicación. No es necesario incluir confirmación.
 - b) Nuevo: El método `optionNew` que crea un nuevo `forest`. Capturen una pantalla significativa.

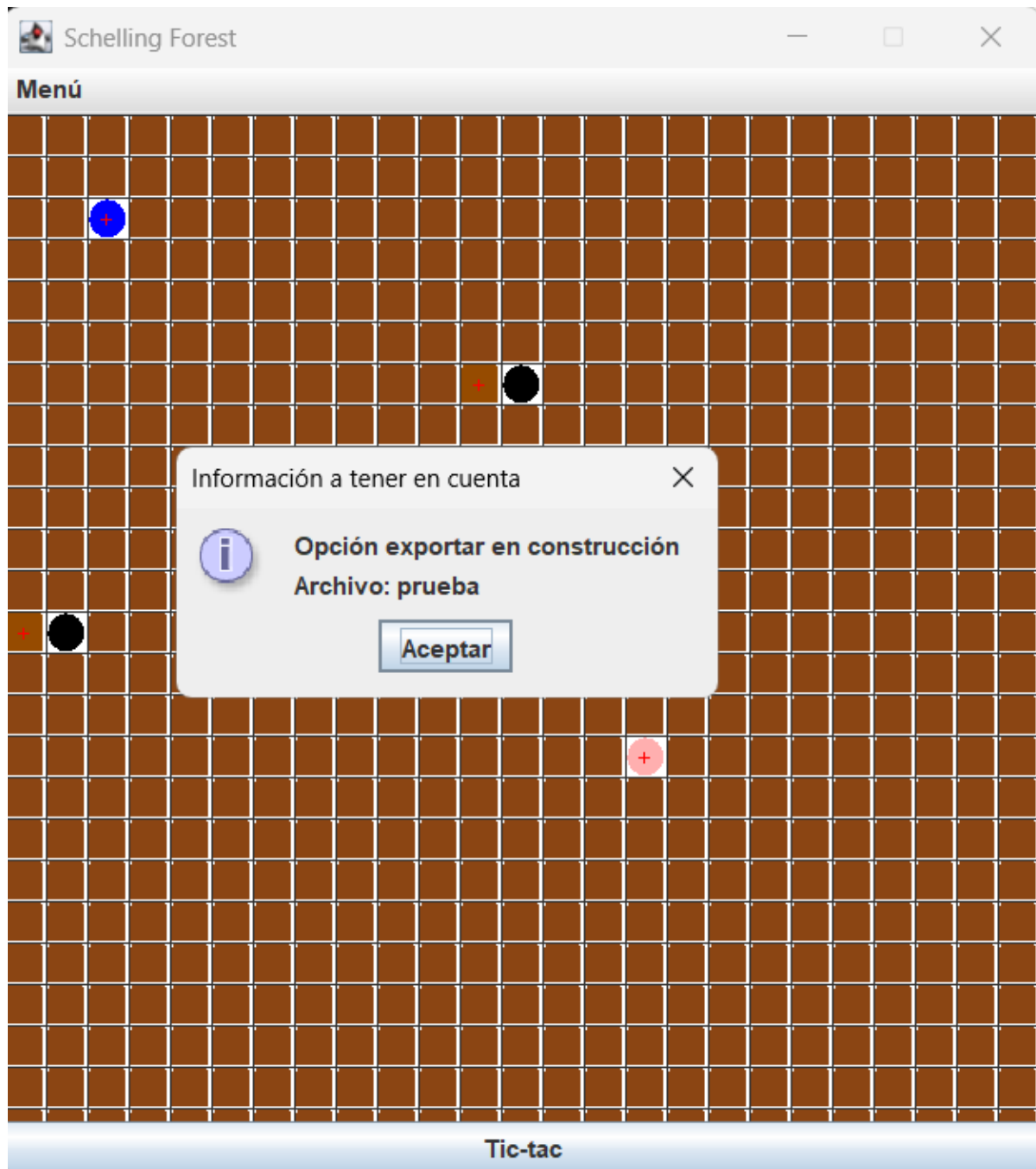


- c) Entrada-salida: Los métodos usan un [FileChooser](#) y atienden la excepción. Estos métodos llaman el método correspondiente del modelo que, por ahora, sólo lanzan una excepción. Ejecuten las diferentes acciones del menú y para cada una de ellas capture una pantalla significativa.









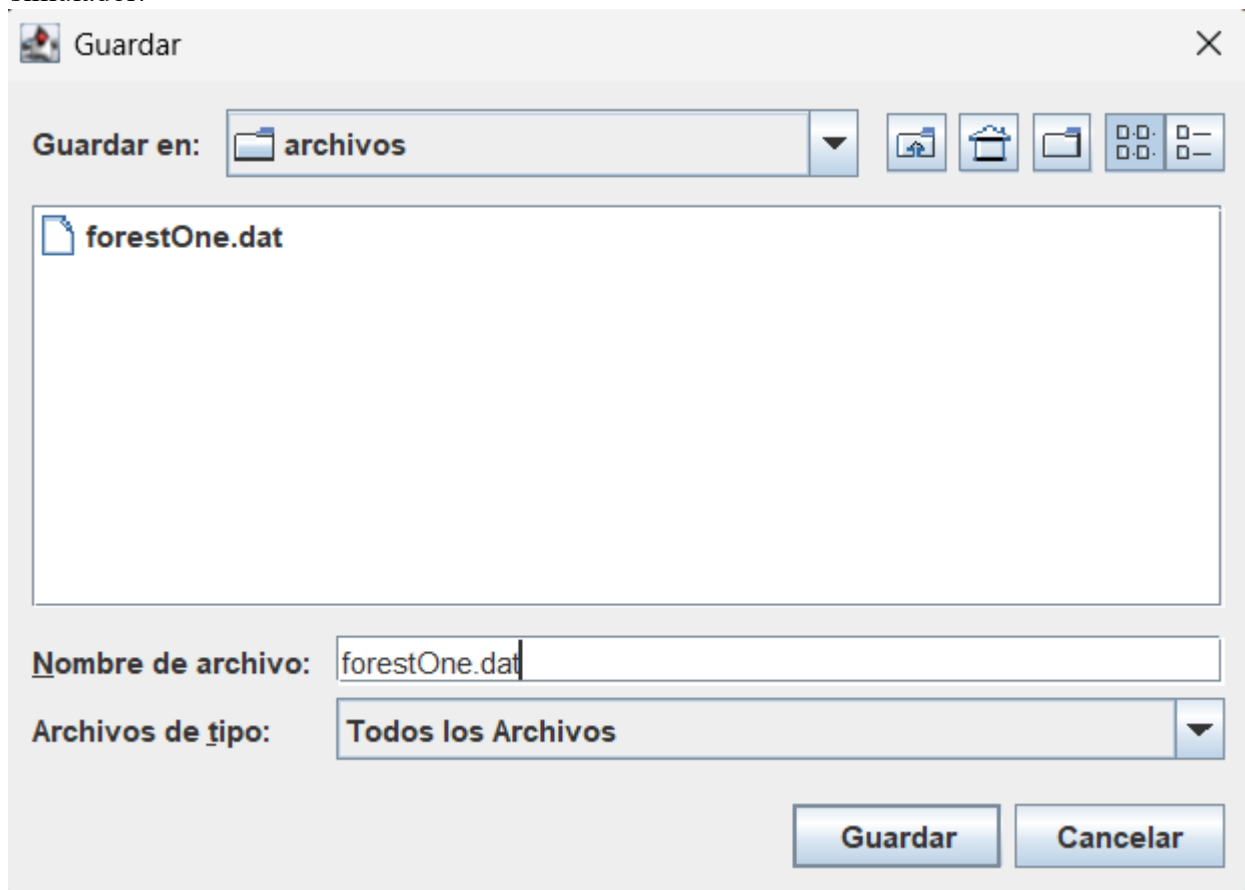
PARTE II. Implementando persistencia [En `lab06.doc forest.astah *.java`] (NO OLVIDEN BDD – MDD)

A Guardar y Abrir

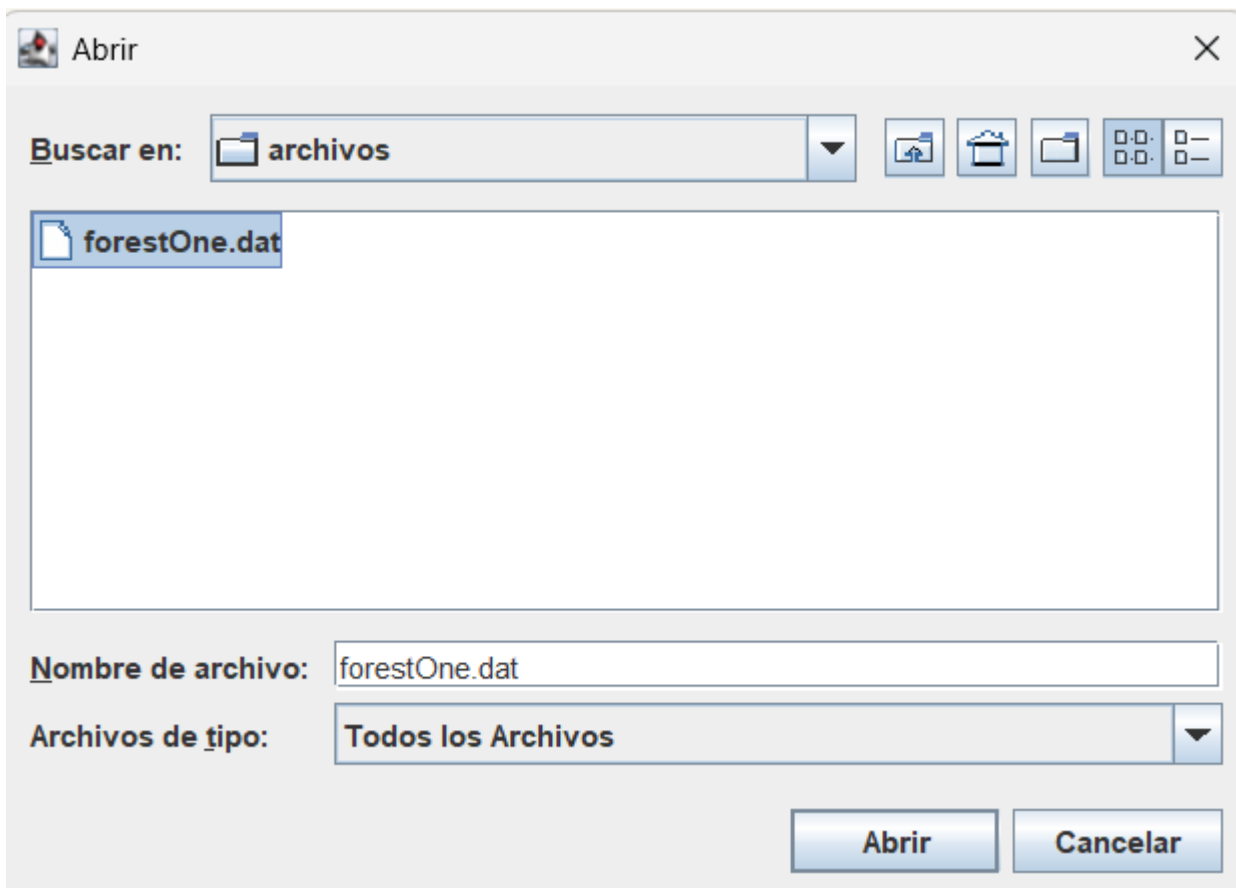
Las opciones guardar y abrir van a ofrecer servicios de persistencia del `forest` como objeto. Los nombres de los archivos deben tener como extensión `.dat`.

1. Copien las versiones actuales de `open` y `saveAs` y renómbrenlos como `open00` y `saveAs00`
2. Construyan el método `saveAs` que ofrece el servicio de guardar en un archivo el estado actual de `forest`. Por ahora para las excepciones sólo consideren un mensaje de error general. No olviden diseño y pruebas de unidad.

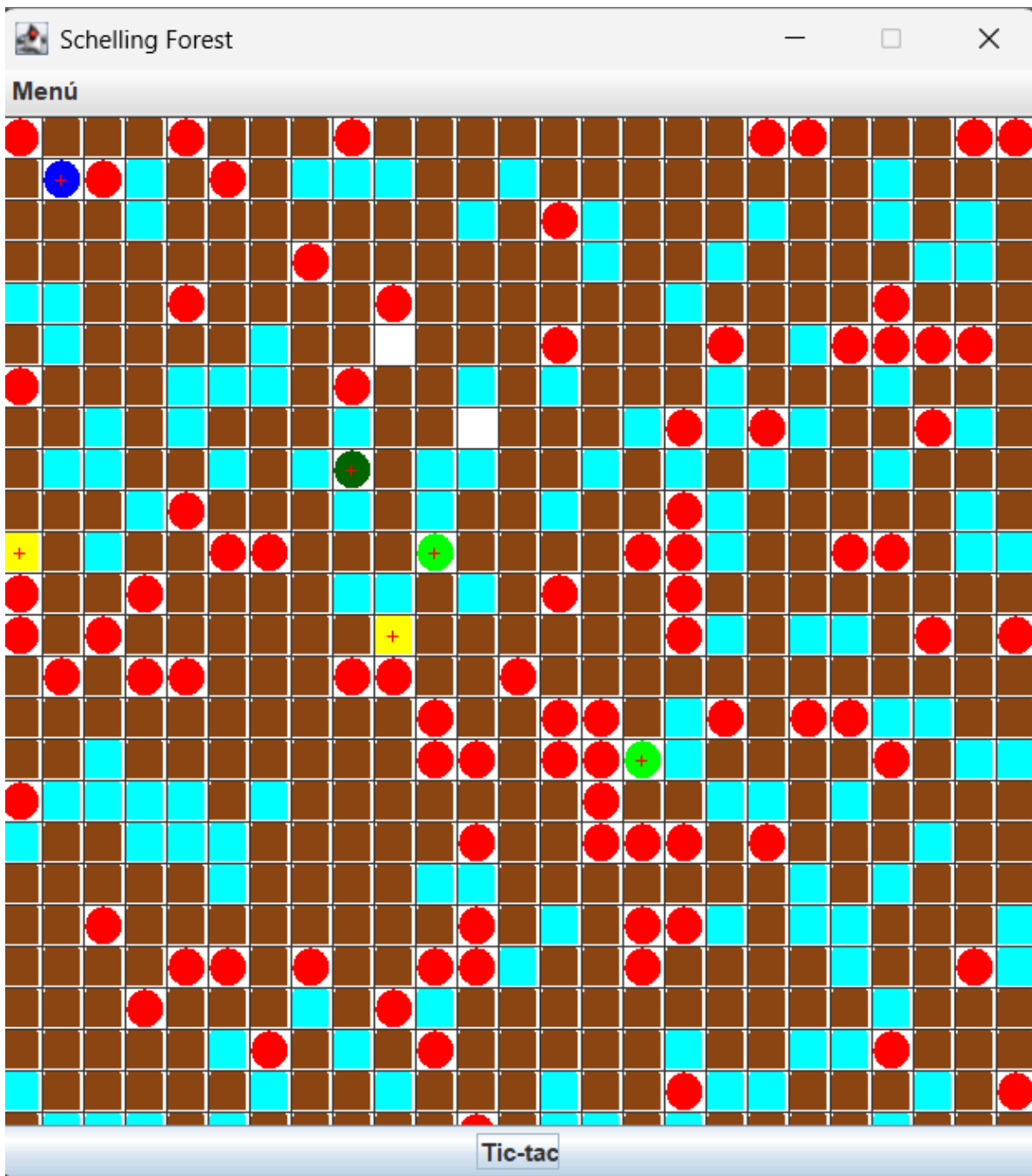
2. Se guardó el archivo “forestOne.dat” en la carpeta “archivos” y seguidamente se cierra el simulador.



3. Se abre el archivo anteriormente guardado.



4. Finalmente se obtiene el archivo a su último guardado (5 Tic-tacs).



B Exportar e Importar

Estas operaciones nos van a permitir importar información de [forest](#) desde un archivo de texto y exportarlo. Los nombres de los archivos de texto deben tener como extensión [.txt](#). Los archivos texto tienen una línea de texto por cada ítem. En cada línea asociada un elemento se especifica el tipo y la posición.

Tree 10, 20

Squirrel 15, 15

1. Copien las versiones actuales de [import](#) y [exportAs](#) y renómbralos como [import00](#) y [exportAs00](#).
2. Construyan el método [exportAs](#) que ofrece el servicio de exportar a un archivo texto el estado actual, con el formato definido. Por ahora para las excepciones sólo consideren un mensaje de error general. No olviden diseño y pruebas de unidad.

3. Realicen una prueba de aceptación de este método: iniciando la aplicación y exportando como [forestOne.txt](#). Editen el archivo y analicen los resultados. ¿Qué pasó?

R/: Se editó la posición del bombero de [2,2] a [3,2]. De manera que, si se intenta abrir el archivo, sucederá un error de java gracias a la Serialización donde el ID del archivo ya no será el mismo, por lo tanto, ya no se podrá cargar ese archivo.dat; así que se debe usar el método import para poder leer un archivo de texto.

4. Construyan el método [import](#) que ofrece el servicio de importar de un archivo texto con el formato definido. Por ahora sólo considere un mensaje de error general. No olviden diseño y pruebas de unidad. (Consulten en la clase [String](#) los métodos [trim](#) y [split](#))

R/:

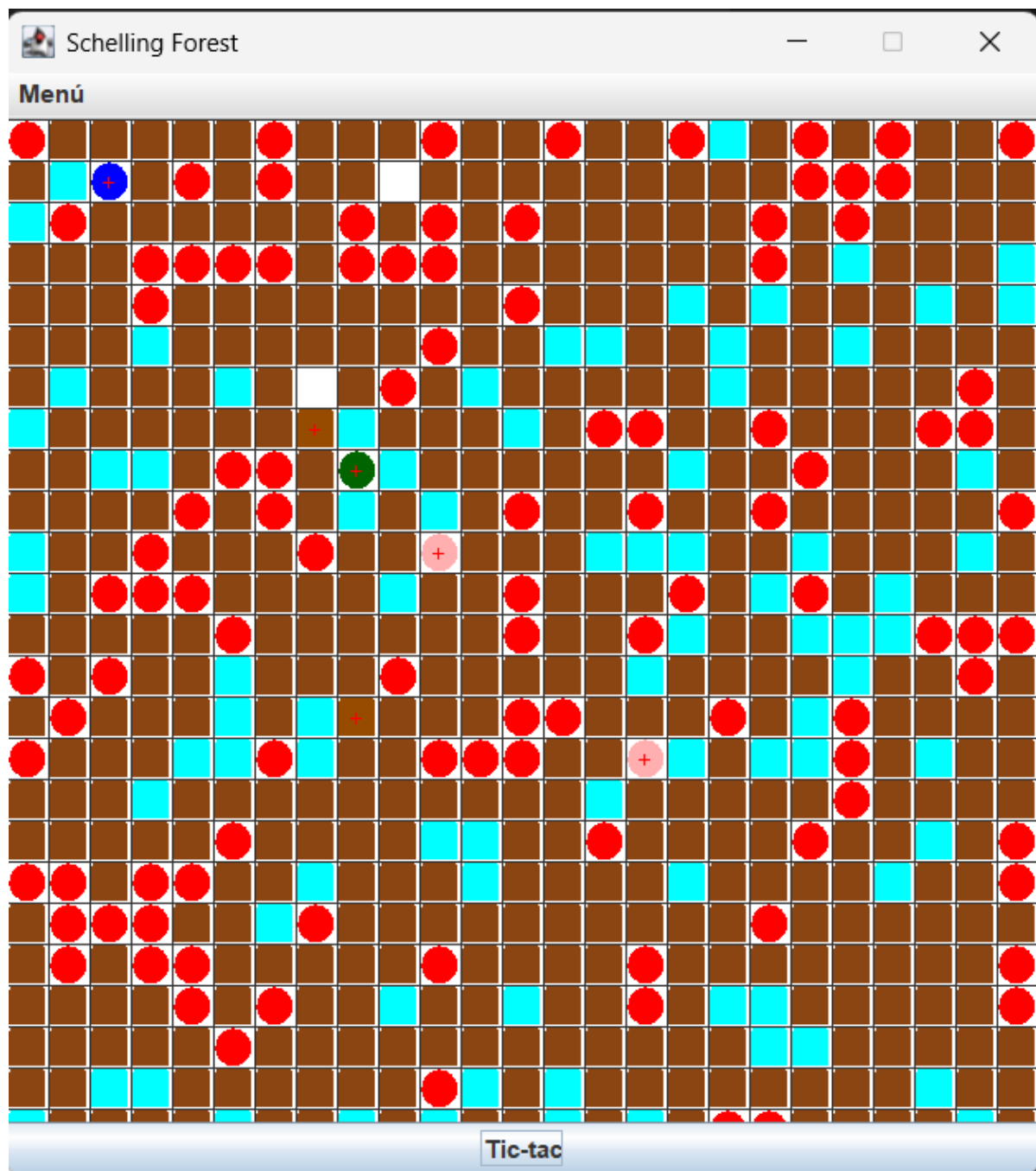
Trim: Su propósito es eliminar espacios en blanco, tabulaciones y saltos de línea al final de una cadena.

Split: Divide una cadena en un arreglo de sub-cadenas.

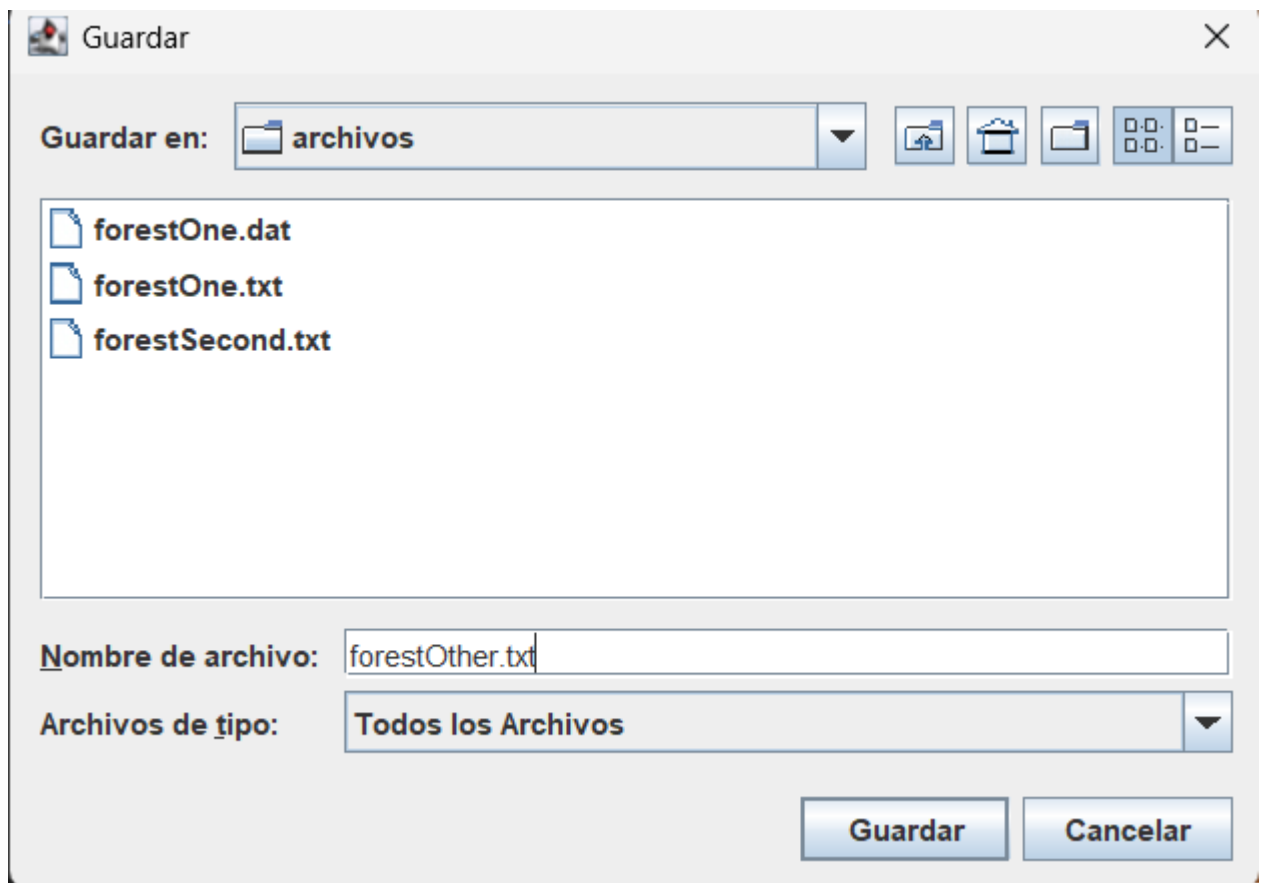
5. Realicen una prueba de aceptación de este par de métodos: iniciando la aplicación exportando a [forestOther.txt](#). saliendo, entrando, creando una nueva e importando el archivo [forestOther.txt](#). ¿Qué resultado obtuvieron? Capturen la pantalla final.

R/:

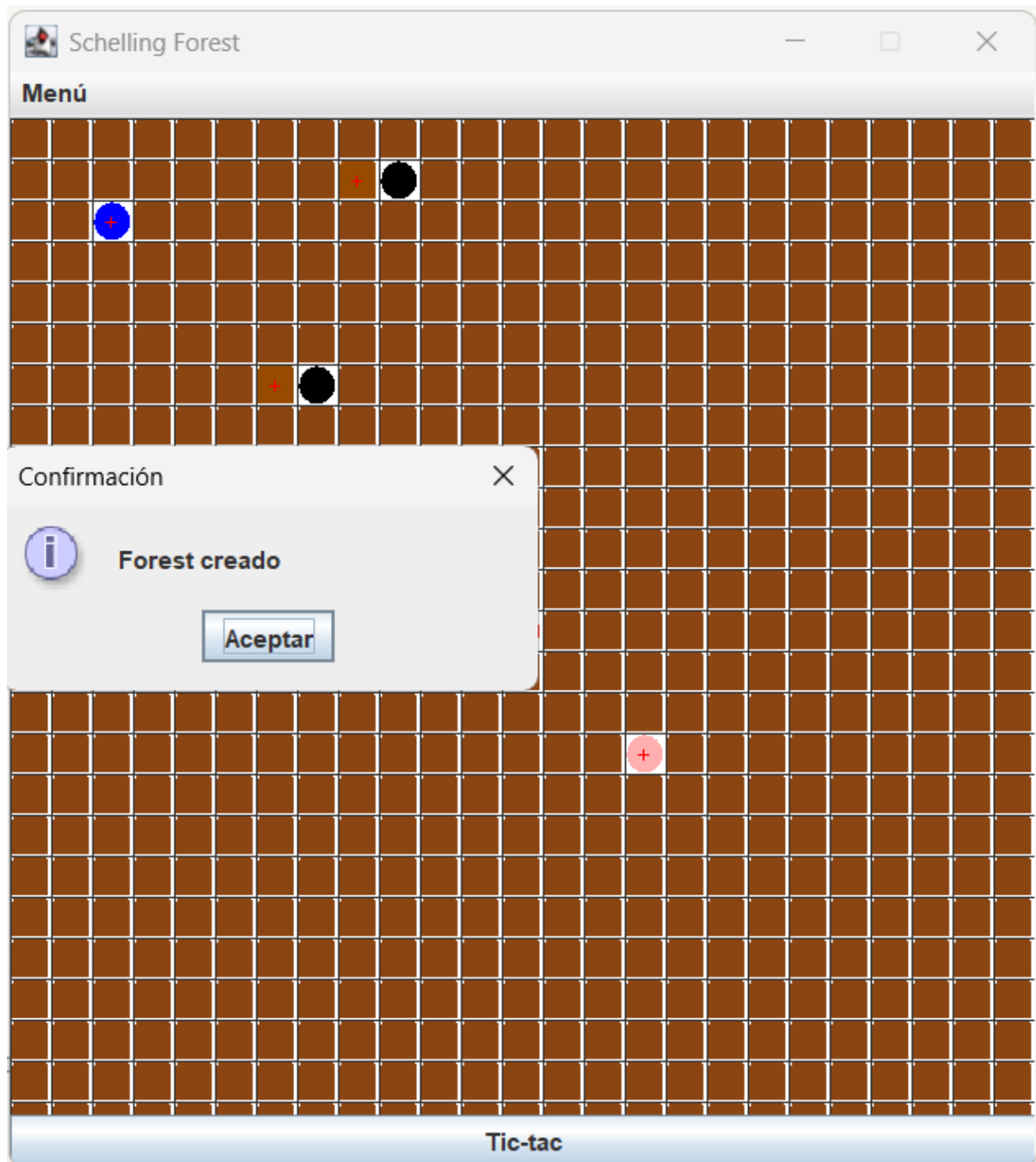
1. Inicio de la aplicación y se hacen 4 clics para ver la diferencia.



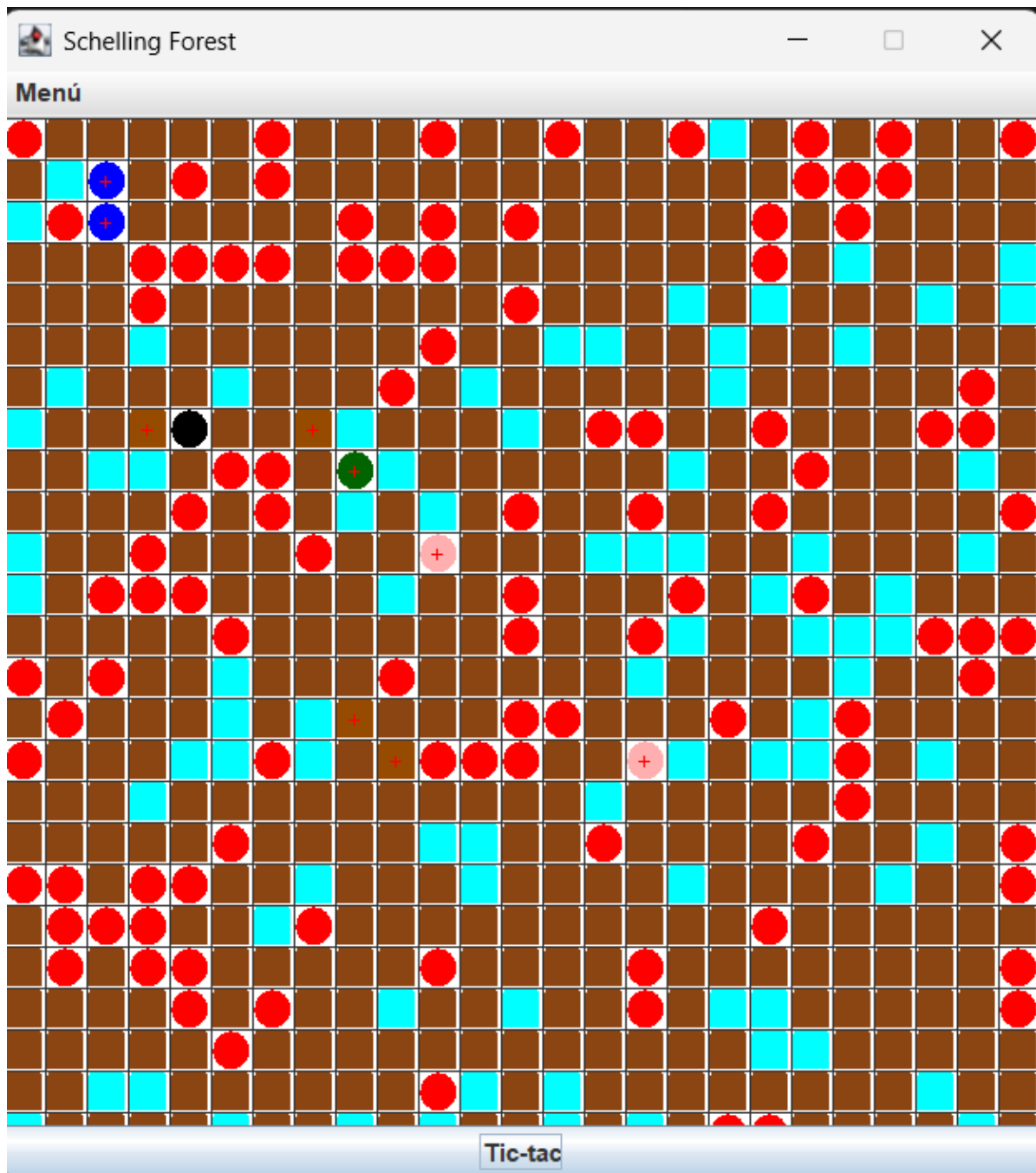
2. Exportar forestOther.txt



3. Se cierra la aplicación y se crea un nuevo Forest



4. Se obtuvo que los objetos estaban en la posición que se dejaron al hacer 4 Tic-tacs.



6. Realicen otra prueba de aceptación de este método escribiendo un archivo de texto correcto en `forestOne.txt` e importe este archivo. ¿Qué resultado obtuvieron? Capturen la pantalla.

R/:

El archivo creado al agregar nuevos objetos fue el siguiente:

```
Tamaño del bosque actual: 20
```

```
Squirrel 4 13
```

```
Water 7 8
```

```
Water 5 4
```

```
Water 6 5
```

```
Tree 10 10
```

```
Fire 11 11
```

```
Pine 5 8
```

```
Pine 1 2
```

```
Pine 9 7
```

```
Pine 7 7
```

```
Pine 17 3
```

```
Firefighter 15 1
```

Este es el resultado:

R/: La diferencia entre el comportamiento 1 es que al abrir un archivo .dat, el método recupera el estado en el que este estaba gracias a la Serialización mientras que el comportamiento 2, al exportar el archivo .txt (en este caso), este método reconstruye los objetos por sí solo. Así mismo, el abrir es más restrictivo, ya que, si el programador llega a cambiar el código y dado que el archivo tiene una posición en memoria en el disco, sus ID's van a ser diferentes; mientras que, al exportar un archivo, no sucede nada si se le cambia alguna línea de código con tal de que el mismo formato que tenga ese archivo coincida con el que lee el método exportar.

PARTE III. Fortaleciendo la persistencia [En lab06.doc forest.astah *.java]

(NO OLVIDEN BDD – MDD)

A Guardar y abrir

1. Copien las versiones actuales de `open` y `saveAs` y renómbrenlos como `open01` y `saveAs01`
2. Perfeccionen el manejo de excepciones de los métodos `open` y `saveAs` detallando los errores. No olviden pruebas de unidad.
3. Realicen una prueba de aceptación para validar uno de los nuevos mensajes diseñados, ejecútenla y capturen la pantalla final.

B Exportar e Importar

1. Copien las versiones actuales de `import` y `exportAs` y renómbrenlos como `import01` y `exportAs01`
2. Perfeccionen el manejo de excepciones de los métodos `import` y `exportAs` detallando los errores. No olviden pruebas de unidad.
3. Realicen una prueba de aceptación para validar uno de los nuevos mensajes diseñados, ejecútenla y capturen la pantalla final.

PARTE IV. BONO. Perfeccionando importar [En lab06.doc forest.astah *.java]

(NO OLVIDEN BDD – MDD)

A Minicompilador

1. Copien las versiones actuales de `import` y `exportAs` y renómbrenlos como `import02` y `exportAs02`
2. Perfeccionen el método `import` para que, además de los errores generales, en las excepciones indique el detalle de los errores encontrados en el archivo (como un compilador) : número de línea donde se encontró el error, palabra que tiene el error y causa de error.
3. Escriban otro archivo con mínimo tres errores diferentes, llámenlo `forestErr.txt`, para ir arreglándolo con ayuda de su “importador”. Presenten las pantallas que contengan los errores.

B Minicompilador flexible

1. Copien las versiones actuales de `import` y `exportAs` y renómbrenlos como `import03` y `exportAs03`
2. Perfeccionen los métodos `import` y `exportAs` para que pueda servir para cualquier tipo de elementos creados en el futuro. No olviden pruebas de unidad. (Investiguen cómo crear un objeto de una clase dado su nombre)
3. Escriban otro archivo de pruebas, llámelo `.txt`, para probar la flexibilidad. Presente una pantalla que contenga un error significativo.

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)
Juan Pablo Cuervo Contreras 28 horas
David Felipe Ortiz Salcedo 28 horas

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
El laboratorio está completo, incluyendo el bono. El código implementa el simulador del bosque, la interfaz gráfica en Swing, el botón Tic-tac, el menú de archivo, apertura y guardado en .dat, importación y exportación en .txt, manejo de excepciones y pruebas JUnit. Además, el bono está cubierto con el importador flexible tipo “minicompilador”, que usa reflexión para crear objetos del paquete domain desde el archivo de texto y reporta varios errores de importación en una sola pasada.
3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?
La práctica XP más útil fue TDD / pruebas unitarias. Fue la más útil porque permitió validar el comportamiento del sistema mientras se agregaban funcionalidades: evolución con ticTac, muerte de ardillas, comportamiento de sombras, guardado/carga .dat, importación/exportación .txt, extensiones inválidas y errores de formato. Esto redujo regresiones al modificar Forest, especialmente en entrada/salida y manejo de excepciones.
4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
El mayor logro fue integrar el modelo del bosque con persistencia, importación/exportación y GUI sin romper la simulación. El sistema no solo muestra entidades como Tree, Squirrel, Pine, Water, Fire, Firefighter, Ground y Shadow, sino que también permite guardar el estado, cargarlo, exportarlo como texto e importarlo de nuevo. Esto hace que el laboratorio pase de ser una simulación básica a una aplicación usable.
5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
El mayor problema técnico fue la entrada/salida de archivos, especialmente la importación desde texto con errores de formato, tipos desconocidos, coordenadas inválidas y extensiones incorrectas. Para resolverlo se creó ForestException, se agregaron validaciones para archivos legibles/escribibles, se separaron errores de lectura, formato y extensión, y se implementó creación dinámica con reflexión usando Class.forName, getConstructor y newInstance.
6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Como equipo hicieron bien la división por responsabilidades: dominio, interfaz, persistencia y pruebas. También mantuvieron versiones anteriores de algunos métodos (open01, exportAs02, importFile03), lo que muestra evolución incremental del laboratorio. Para mejorar, se comprometen a escribir pruebas más deterministas para clases con aleatoriedad como Squirrel, Ground y Firefighter, limpiar artefactos compilados del repositorio y revisar nombres/codificación de comentarios para mantener mayor calidad.
7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

W3Schools. — *Tutoriales y referencias de programación*. <https://www.w3schools.com/>